

## CHAPITRE 3

# Fonctionnement du système et développement logiciel

---

Ce chapitre décrit comment le système **DisrtuCare** fonctionne réellement, une fois assemblé. Après avoir présenté les composants matériels au chapitre précédent, nous expliquons ici **ce qui se passe quand le système est en marche** : comment le distributeur délivre une pilule à l'heure prévue, comment l'application mobile a été conçue, comment les deux communiquent, et enfin comment toute la chaîne se déroule « du réglage de l'heure par l'utilisateur jusqu'à la pilule dans le plateau ». Le propos reste volontairement **accessible** : les notions informatiques sont expliquées par analogie plutôt que par le détail du code.

**Vue en une phrase.** DisrtuCare associe un **distributeur autonome** (un petit ordinateur « NodeMCU » qui pilote un moteur, une horloge et une LED) à une **application mobile compagnon** qui sert de tableau de bord, de carnet de suivi et de télécommande. Les deux dialoguent par **WiFi**, sur le réseau domestique du patient.

## 3.1 Fonctionnement du système matériel

### 3.1.1 Le principe général

Le cœur du distributeur est un **NodeMCU V3**, un microcontrôleur de la famille ESP8266. C'est l'équivalent d'un très petit ordinateur : il exécute en boucle un programme (le *firmware*) et intègre nativement le **WiFi**, ce qui lui permet de communiquer avec le téléphone sans câble ni module supplémentaire. Autour de lui s'articulent quatre organes :

| Organe                                                              | Rôle physiologique du système                                                                                  |
|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Moteur pas à pas<br><b>28BYJ-48</b> (via le pilote <b>ULN2003</b> ) | Le « muscle » : il fait tourner le carrousel de pilules d'un compartiment, libérant la dose.                   |
| Horloge temps réel<br><b>DS3231</b>                                 | La « montre » : elle garde l'heure exacte, même après une coupure de courant, grâce à une pile bouton.         |
| <b>LED</b> bleue                                                    | Le « signal d'alerte » : elle clignote pour signaler qu'une dose vient d'être délivrée et attend confirmation. |
| <b>Bouton</b> poussoir                                              | Le « retour patient » : en appuyant, l'utilisateur confirme qu'il a bien pris sa pilule.                       |
| Écran <b>LCD</b> 16×2                                               | L'« afficheur » : il montre l'heure, l'horaire et l'adresse de connexion au démarrage.                         |

### 3.1.2 Le moteur : comment une pilule est libérée

La distribution repose entièrement sur la **rotation précise** du moteur. Le 28BYJ-48 est un *moteur pas à pas* : au lieu de tourner librement, il avance par petits « pas » commandables un par un, ce qui permet de maîtriser exactement de combien il tourne. Dans DisrtuCare :

- Un tour complet du moteur correspond à **4096 demi-pas**.
- Le carrousel possède **4 compartiments** ; un compartiment correspond donc à  $4096 \div 4 = 1024$  **demi-pas**.
- Chaque demi-pas dure 2 millisecondes, soit **environ 2 secondes** de rotation pour amener le compartiment suivant au-dessus de l'ouverture.
- Dès la rotation terminée, le programme **coupe l'alimentation des bobines** du moteur, pour éviter qu'il chauffe inutilement à l'arrêt.

**Pourquoi un moteur « pas à pas » ?** Parce qu'on a besoin d'une précision médicale : il faut délivrer *exactement une* dose, ni plus ni moins. Compter les pas garantit que le carrousel s'arrête pile sur le compartiment suivant.

### 3.1.3 L'horloge : savoir quand distribuer

Pour distribuer « à 8h00 », le système doit connaître l'heure. C'est le rôle du module **DS3231**, une horloge alimentée par une petite pile qui continue de compter le temps même appareil éteint. Le firmware a été conçu avec une **sécurité intelligente** : si l'horloge matérielle n'est pas présente, il bascule sur une **horloge logicielle de secours**, mise à l'heure par le téléphone lors de la connexion. Le système reste donc fonctionnel même sans le module DS3231.

Tant que l'heure n'est pas connue de façon fiable, le programme **interdit toute distribution** (fonction `rtcIsRunning`). Cela évite qu'une dose parte par erreur à minuit, heure par défaut d'un appareil non réglé.

### 3.1.4 La boucle de fonctionnement permanente

Une fois allumé, le distributeur exécute sans arrêt une **boucle** qui enchaîne quatre vérifications, plusieurs fois par seconde et **sans jamais se bloquer** :

1. **Répondre au téléphone** : traiter les éventuelles demandes de l'application.
2. **Vérifier l'horaire** : l'heure actuelle correspond-elle à la dose du matin ou du soir ?
3. **Surveiller le bouton** : l'utilisateur vient-il de confirmer une prise ?
4. **Animer la LED** : la faire clignoter tant qu'une confirmation est attendue.

C'est ce fonctionnement **autonome** qui fait la robustesse du dispositif : même si le téléphone est éteint ou hors réseau, le distributeur connaît l'heure, délivre les doses programmées et attend la confirmation du patient.

### 3.1.5 La mémoire permanente (EEPROM)

L'horaire (heure du matin, heure du soir) et le nom du médicament sont écrits dans une **mémoire non volatile** appelée EEPROM. Concrètement, ces réglages **survivent à une coupure de courant** : après un redémarrage, le distributeur retrouve son horaire sans avoir besoin du téléphone. Un « octet témoin » permet au programme de savoir si la mémoire a déjà été initialisée.

## 3.2 Développement logiciel

Le logiciel de DisrtuCare se compose de **deux programmes distincts** qui coopèrent : le **firmware embarqué** (décrit ci-dessus, écrit en langage Arduino / C++) et l'**application mobile**. Cette section détaille la conception de l'application.

### 3.2.1 La technologie choisie et pourquoi

L'application a été développée avec **Expo / React Native**, une technologie qui permet d'écrire **une seule fois** le code et de le faire fonctionner aussi bien sur **Android** que sur **iOS**. Le langage utilisé est le **TypeScript**. Ce choix évite de développer deux applications séparées et accélère donc nettement la réalisation.

| Brique logicielle              | À quoi elle sert                                                                                               |
|--------------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>Expo / React Native</b>     | Construire l'interface et la faire tourner sur téléphone.                                                      |
| <b>expo-sqlite</b>             | Une base de données locale, dans le téléphone : l'app fonctionne <b>hors-ligne</b> , sans serveur ni internet. |
| <b>expo-notifications</b>      | Programmer des <b>rappels</b> (notifications) aux heures de prise.                                             |
| <b>react-native-reanimated</b> | Rendre les animations fluides et agréables.                                                                    |

### 3.2.2 L'organisation de l'application en écrans

L'interface est organisée en **quatre onglets**, pensés pour une utilisation simple, notamment par des **personnes âgées** :

| Écran                  | Fonction                                                                                                                                                     |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tableau de bord</b> | Affiche la <b>prochaine dose</b> (avec compte à rebours), un <b>anneau d'observation</b> sur 30 jours, et les doses du jour (matin / soir) avec leur statut. |
| <b>Historique</b>      | Journal jour par jour des prises, avec un code couleur.                                                                                                      |
| <b>Appareil</b>        | Permet de <b>se connecter au distributeur</b> (saisie de son adresse), et offre des boutons de test (moteur, LED, synchronisation).                          |
| <b>Réglages</b>        | Définit les <b>heures</b> matin et soir, le <b>nom du médicament</b> , et active / désactive les notifications.                                              |

### 3.2.3 La base de données locale

Toutes les informations sont conservées **dans le téléphone**, dans une petite base de données SQLite. Elle comporte deux tables :

- **schedule** (les réglages) : une seule ligne contenant le nom du médicament, les heures du matin et du soir, l'état des notifications et l'adresse du distributeur.
- **dose\_logs** (le carnet de prises) : une entrée par dose, avec la date, le type (matin / soir) et le statut : **prise** / **en retard** / **manquée**. Une règle garantit **une seule entrée matin et une seule entrée soir par jour**.

### 3.2.4 Les rappels (notifications)

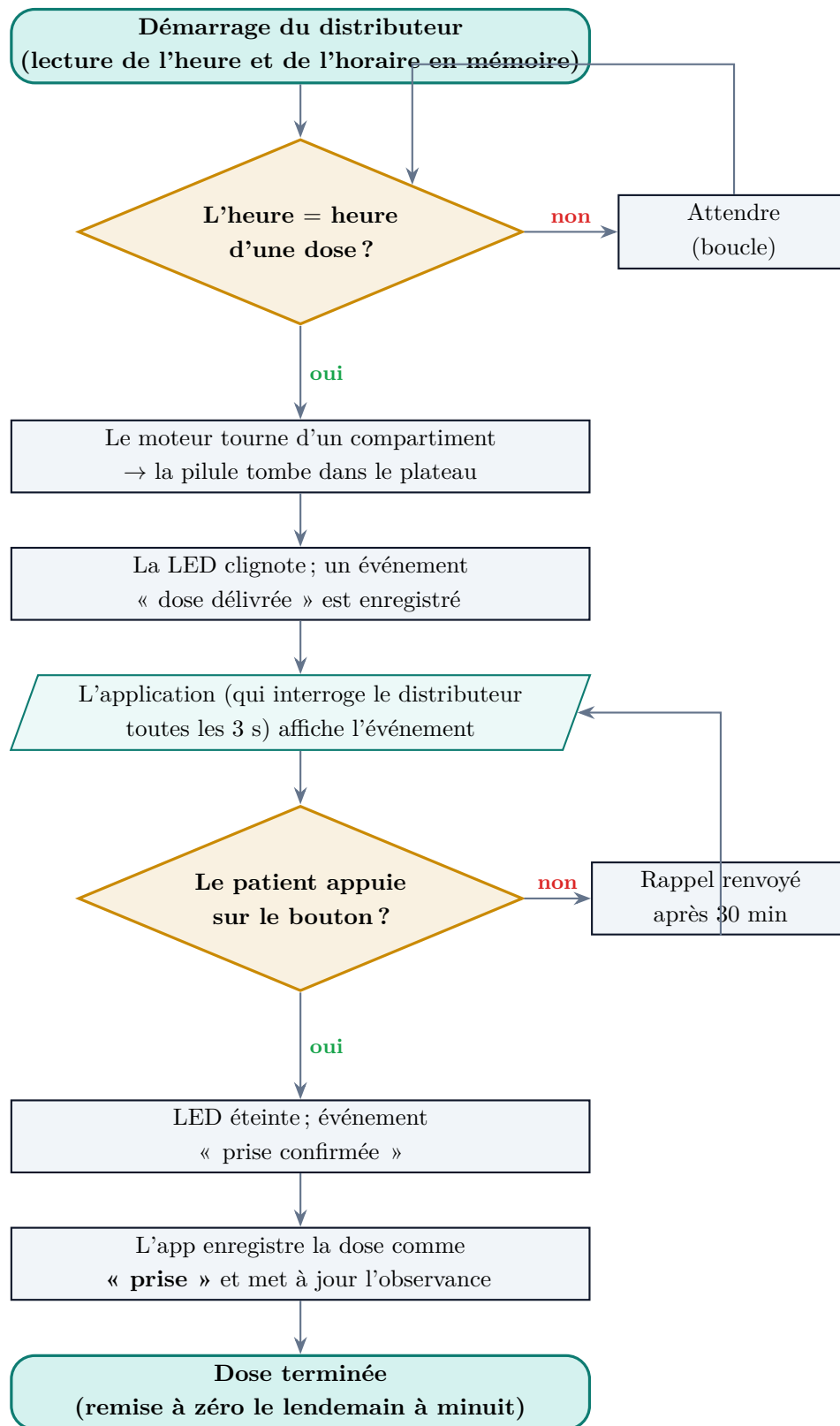
À chaque enregistrement de l'horaire, l'application **reprogramme automatiquement quatre rappels quotidiens** : un rappel le matin, un le soir, et deux **relances 30 minutes plus tard** si la prise n'a pas été confirmée. Ces rappels sont **locaux** : ils s'affichent sur le téléphone même sans connexion internet.

### 3.2.5 L'accessibilité : une exigence de conception

Le public cible étant les **patients âgés**, l'interface a été conçue avec : grandes polices (18 à 22 px), **fort contraste** (fond bleu marine, accent turquoise), **grandes zones tactiles** (au moins 56 px) et un **code couleur intuitif** (vert / jaune / rouge) pour le statut de chaque dose.

## 3.3 Organigramme (logigramme) du fonctionnement

L'organigramme ci-dessous résume le **cycle complet d'une dose**, depuis la vérification de l'heure par le distributeur jusqu'à l'enregistrement de la prise dans l'application. Les losanges représentent des **décisions** ; les rectangles, des **actions**.



**Lecture de l'organigramme.** Tant que l'heure ne correspond pas, le distributeur « tourne en rond » (boucle d'attente). Au moment voulu, il agit : il distribue, signale, et attend la confirmation du patient. Si le patient n'appuie pas, l'application le relance ; quand il appuie, la prise est enregistrée automatiquement.

### 3.4 Intégration matériel–logiciel

C'est le point clé du projet : faire **dialoguer** le distributeur et le téléphone.

#### 3.4.1 Le moyen de communication : le WiFi

Le distributeur et le téléphone sont connectés au **même réseau WiFi domestique**. Le NodeMCU se comporte alors comme un **petit serveur web** : il possède une **adresse** (par exemple 192.168.11.109) que l'utilisateur saisit dans l'onglet **Appareil** de l'application. Cette adresse s'affiche sur l'écran LCD au démarrage, puis elle est **mémorisée** par l'application pour les fois suivantes.

#### 3.4.2 Le langage commun : des messages HTTP/JSON

Les deux appareils échangent de courts messages au format **JSON** (un format texte simple et lisible) via le protocole **HTTP**, le même que celui des sites web. Cinq « adresses » (endpoints) sont définies sur le distributeur :

| Message  | Signification                                               |
|----------|-------------------------------------------------------------|
| /status  | « Donne-moi ton état » (heure, horaire, dernier événement). |
| /sync    | « Voici le nouvel horaire » (matin, soir, médicament).      |
| /settime | « Voici l'heure exacte » (pour régler l'horloge).           |
| /led     | « Allume ou éteins la LED ».                                |
| /motor   | « Fais tourner le moteur » (bouton de test).                |

#### 3.4.3 Le sens distributeur → application : le « sondage »

Un microcontrôleur ne peut pas facilement « appeler » le téléphone de lui-même. La solution retenue est simple et robuste : c'est l'application qui **interroge le distributeur toutes les 3 secondes** (mécanisme dit de *polling*). Chaque réponse contient un **numéro d'événement**. Quand ce numéro change, l'application comprend qu'un événement vient de se produire (une dose délivrée, une prise confirmée) et réagit en conséquence.

**Une analogie.** Plutôt que le distributeur « téléphone » à l'app, c'est l'app qui demande régulièrement : « Du nouveau ? ». Tant que le numéro d'événement est le même, rien n'a changé ; dès qu'il augmente, l'app sait qu'un fait nouveau s'est produit.

#### 3.4.4 Le pipeline complet : du réglage à la prise

Voici le scénario de bout en bout, qui relie tout ce qui précède :

**1. Réglage (utilisateur).** Dans l'écran Réglages, l'utilisateur définit 08 :00 / 20 :00. L'application : (a) enregistre l'horaire dans sa base locale ; (b) l'envoie au distributeur via /sync (qui le sauve en EEPROM et l'affiche sur le LCD) ; (c) programme les notifications de rappel.

**2. Distribution (matériel).** À 08 :00, le distributeur détecte que l'heure correspond à la dose du matin. Le moteur tourne d'un compartiment, la pilule tombe dans le plateau, la LED se met à clignoter, et un événement « dose délivrée » est enregistré.

**3. Notification (logiciel).** Au même moment, le téléphone affiche son rappel local « C'est l'heure de votre dose du matin ». En interrogeant le distributeur, l'application voit le nouvel événement et l'affiche dans son journal.

**4. Confirmation (utilisateur + matériel).** Le patient prend sa pilule et **appuie sur le bouton** du distributeur. La LED s'éteint, et un événement « prise confirmée » est enregistré.

**5. Traçabilité (logiciel).** L'application capte la confirmation et **enregistre automatiquement la dose comme « prise »** dans le carnet, sans aucune action manuelle. L'**anneau d'observance** (30 jours) se recalcule.

**Le point fort de l'intégration :** un simple **geste physique** sur le distributeur (appuyer sur le bouton) **met à jour le dossier numérique** du patient. Le patient n'a rien à saisir ; le suivi d'observance se construit tout seul.

### 3.5 Difficultés rencontrées et solutions apportées

La réalisation a soulevé plusieurs difficultés concrètes, matérielles comme logicielles. Les principales sont résumées ci-dessous.

| Difficulté                                                                                                                  | Solution apportée                                                                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Le moteur ne tournait pas</b> malgré un câblage en apparence correct.                                                    | Les fils de commande étaient branchés sur les broches D1–D4 alors que le programme pilotait D5–D8. Le <b>déplacement des fils sur D5–D8</b> a résolu le problème ; ces broches évitent en plus les broches « sensibles au démarrage » de l’ESP8266. |
| <b>Alimentation insuffisante du moteur.</b> Le moteur était relié à une broche n’ayant pas la tension nécessaire.           | Reraccordement sur la broche <b>5V (VU)</b> reliée directement à l’USB, et installation du <b>cavalier d’alimentation</b> du pilote ULN2003.                                                                                                        |
| <b>Coupures du microcontrôleur</b> (« brownout ») au démarrage du moteur, qui tirait trop de courant via le port USB du PC. | Débrancher l’alimentation moteur pendant le téléversement ; la solution définitive est une <b>alimentation 5V séparée</b> pour le moteur, avec une masse commune.                                                                                   |
| <b>Impossible de connaître l’adresse du distributeur</b> sans écran.                                                        | Le firmware <b>affiche l’adresse sur le LCD</b> pendant 5 secondes au démarrage (et l’imprime aussi sur le port série pour le débogage).                                                                                                            |
| <b>Risque de distribution parasite</b> si l’heure n’est pas connue.                                                         | Le programme <b>interdit toute distribution tant que l’horloge n’est pas valide</b> , évitant un déclenchement à minuit par défaut.                                                                                                                 |
| <b>Fonctionnement sans module d’horloge (DS3231).</b>                                                                       | Mise en place d’une <b>horloge logicielle de secours</b> , resynchronisée par le téléphone à chaque connexion.                                                                                                                                      |
| <b>Microcontrôleur incapable de notifier le téléphone</b> directement.                                                      | Adoption du <b>sondage par numéro d’événement</b> : l’app interroge le distributeur toutes les 3 s, sans technologie complexe (pas de WebSocket).                                                                                                   |
| <b>Évolution du cahier des charges</b> : connectivité d’abord interdite, puis exigée.                                       | Migration d’une approche <b>Bluetooth</b> vers une approche <b>WiFi/HTTP</b> , plus simple à tester et compatible avec l’environnement Expo Go (sans module natif).                                                                                 |

### 3.5.1 Limites connues et pistes d'amélioration

- **Communication non chiffrée et sans authentification** : sur le réseau local, toute personne connectée pourrait commander le moteur. Acceptable pour un **prototype**, à sécuriser pour une version finale.
- **Identifiants WiFi inscrits en clair** dans le code du firmware : à externaliser dans un fichier de configuration protégé.
- **Mémoire d'événements limitée** : un seul événement est conservé entre deux interrogations. Si deux événements surviennent en moins de 3 secondes, le premier peut être perdu.

## Conclusion du chapitre

Ce chapitre a montré que DisrtuCare repose sur une architecture **découplée mais coordonnée** : un distributeur **autonome** qui assure la fonction vitale (délivrer la bonne dose à la bonne heure) et une application **compagnon** qui enrichit l'expérience (rappels, suivi d'observance, télécommande). Leur intégration, via une communication WiFi simple et robuste, permet qu'un **geste physique** du patient se traduise automatiquement en **donnée de suivi**. Les difficultés rencontrées, essentiellement matérielles, ont toutes trouvé une solution, et les limites restantes sont clairement identifiées en vue d'une version aboutie.